

Engineering Documentation: AI/ML Phase 1, Week 3

Document Status: Approved & Completed

Author: Jainish Mehta

Focus: Applied Linear Algebra, Tensor Architecture, Matrix Arithmetic, and Computer Vision.

Executive Summary

The objective of Week 3 was to transition from Data Engineering (SQL) into the core mathematical engine of Artificial Intelligence. Machine Learning models do not process raw text or images; they process massive grids of numbers using Linear Algebra. This week focused on deprecating slow Python `for` loops in favor of hardware-accelerated **NumPy Tensors**. The curriculum covered dimensionality, Vectorization, Broadcasting, the Dot Product, Matrix Inversion, and culminated in building an MLOps Polynomial Regression model and a Computer Vision image processor from scratch.

Technology Stack

- **IDE:** Visual Studio Code (VS Code)
- **Terminal:** Git Bash (Linux-based CLI environment)
- **Version Control:** Git & GitHub
- **Language:** Python 3.x
- **Libraries:** `numpy`, `matplotlib`, `sqlite3`, `Pillow` (PIL)

Daily Execution Log

Day 1 & 2: Tensor Architecture & Hardware Optimization

- **Objective:** Migrate to an Enterprise Monorepo environment and establish contiguous memory data structures.
- **What was done:**
 - Upgraded the repository architecture to a centralized Monorepo virtual environment (`.venv`).

- Initialized mathematical arrays across all AI dimensional planes: Scalars (0D), Vectors (1D), Matrices (2D), and Tensors (3D).
- Implemented shape telemetry using `.shape` and `.ndim` attributes.
- **Why we did it (The Rationale):**
 - Native Python Lists store scattered pointers in memory, causing severe CPU Cache misses. NumPy arrays allocate contiguous blocks of C-level memory. This allows the CPU to process data up to 50x faster, which is a fundamental requirement for scaling Machine Learning models.

Day 3 & 4: Vectorization, Matrix Algebra & Shape Shifting

- **Objective:** Execute hardware-accelerated mathematics and resolve structural dimensional clashing.
- **What was done:**
 - Executed element-wise arithmetic utilizing **Vectorization** and **Broadcasting** (dynamically stretching arrays without duplicating data in RAM).
 - Computed artificial neuron activations using the **Dot Product** (`np.dot`).
 - Executed **Matrix Multiplication** (`@`), adhering strictly to the Inner Dimensions Rule.
 - Manipulated architectural constraints via **Transposition** (`.T`) and data flattening (`.reshape(-1)`).
- **Why we did it (The Rationale):**
 - The core engine of Deep Learning is $A \times X = B$. If the shape (dimensionality) of an input tensor does not perfectly align with the shape of the model's weight matrix, the architecture will fatally crash. Transposition and Reshaping are the primary diagnostic skills required to prevent these pipeline failures.

Day 5 & 6: Inverse Matrices & The Normal Equation

- **Objective:** Solve linear systems programmatically without relying on algebraic division.
- **What was done:**
 - Generated geometric anchors using the Identity Matrix (`np.eye`).
 - Computed algorithmic division utilizing Matrix Inversion and Pseudo-Inversion (`np.linalg.pinv`).
 - Mathematically proven the inversion by multiplying a matrix by its inverse to yield the Identity Matrix, safely handling floating-point anomalies (-0.0).

- **Why we did it (The Rationale):**

- Matrix division does not exist in linear algebra. To calculate the exact optimal weights for Ordinary Least Squares (OLS) Linear Regression without iterative guessing, an engineer must invert the feature matrix. This is the exact math running beneath the hood of libraries like Scikit-Learn.

Day 7: MLOps Regression Engine & Computer Vision (Projects)

- **Objective:** Apply Linear Algebra to real-world predictive modeling and unstructured data processing.
- **What was done:**
 - **Project 1 (Adaptive MLOps AI):** Architected an MVC (Model-View-Controller) Python application. Used Polynomial Features (X^2) and the Normal Equation to train an AI that mathematically bends its line of best fit. Integrated an SQLite database to log `predicted_price` vs `actual_price` telemetry.
 - **Project 2 (NumPy Computer Vision):** Loaded a high-definition RGB `.jpg` image as a 3D Tensor `(H, W, 3)`. Executed a Matrix Dot Product against Luma coding weights to mathematically crush the 3 color channels into a single 2D Grayscale matrix.
- **Why we did it (The Rationale):**
 - Proves that advanced linear algebra powers both predictive modeling (regression curves) and unstructured data processing (computer vision preprocessing). Splitting the code into `model.py`, `visualize.py`, and `main.py` enforced Separation of Concerns (SoC), cementing Senior Software Engineering standards.

Key Deliverables Achieved

1. **The Monorepo Pipeline:** A centralized, enterprise-grade virtual environment routing dynamic system paths via `pathlib`.
2. **Adaptive MLOps Regression Engine:** A continuous-learning AI model that tests its own accuracy, adapts its polynomial curve, and logs its failures into an SQL database.
3. **Computer Vision Tensor Transformer:** A mathematical image-processing engine built completely from scratch without using OpenCV or deep learning libraries.

Next Steps (Week 4 Preview)

With the deterministic mathematics of Linear Algebra secured, the focus shifts to handling uncertainty in **Week 4: Applied Probability & Statistics**. Week 4 will cover Gaussian (Normal) Distributions, Variance, Z-Scores, and Bayes Theorem, establishing the statistical foundation required to build Outlier Detection systems and Spam Classifiers.